Assignment 004: Personal Diary

Program Report

Zheng Yifeng | Student ID: 3250105856

1 Introduction

Personal Diary is a command-line diary made of four programs: `pdadd`, `pdlist`, `pdshow` and `pdremove`. They use the same classes and the same data file, `diary.txt`, but each one is compiled into its own executable and never calls another program. They work together only through the data file, redirection, and the script `pd.sh`.

The programs were built and tested on Ubuntu 24.04 (Linux) with g++ 13.3.0, C++17 and bash.

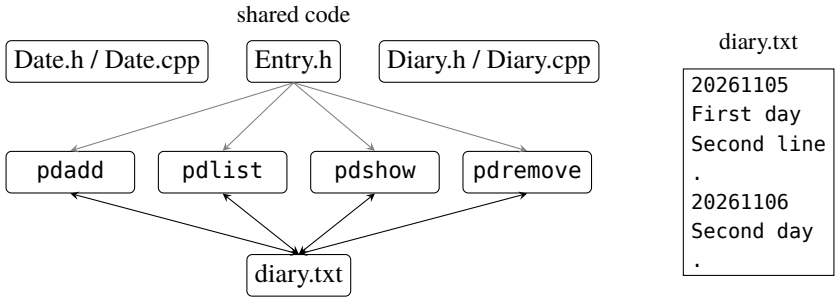
2 Program Design

Date stores the year, month and day. `Date::parse` checks a `YYYYMMDD` string, including the length of each month and leap years. `toString` turns it back into eight digits with leading zeros, and `operator<` lets `std::map` sort the entries by date.

Entry is one diary entry: a `Date` and its lines of text (`vector<string>`). It only holds data, so it has no `.cpp` file.

Diary keeps all entries in `std::map<Date, Entry>`. The map keeps the dates in order and allows only one entry for each date, so `add` can simply replace an old entry. `Diary` also provides `remove`, `find`, `listall`, `between`, `load` and `save`.

Every program follows the same steps: check the arguments, parse the date, load `diary.txt`, do its own job, save the file, and return an exit code. Results are written to `stdout`, and every message is written to `stderr`, so the output can be redirected without extra text.



In `diary.txt`, each entry is a date line, its content, and then a line with a single "`.`". This is the same rule that `pdadd` uses to read its input. Since `pdadd` stops at such a line, the content of an entry can never contain it, so the dot safely marks the end of each entry.

Command	Behaviour	Exit code
`pdadd <date>`	Reads lines from `stdin` until a single "`.`" or EOF; replaces an entry of the same date	0 / 1
`pdlist [<start> <end>]`	Lists entries in date order, one line each: date, a tab, and the first line; the range includes both ends	0 / 1
`pdshow <date>`	Prints only the content, so it can be redirected to `pdadd`	0 / 1
`pdremove <date>`	Removes the entry of that date	0 / −1

3 Running the Programs

Run these commands in the `code` directory. `diary.txt` is created in the current directory.

```
for p in pdadd pdlist pdshow pdremove; do
    g++ -std=c++17 -Wall -Wextra $p.cpp Date.cpp Diary.cpp -o $p; done
bash pd.sh pdadd 20261105 < input.txt
bash pd.sh pdlist 20261101 20261130
bash pd.sh pdshow 20261105 > copy.txt
bash pd.sh pdremove 20261105
```

`pd.sh` runs the program named by its first argument and passes on the other arguments and `stdin`. Because the script uses `set -e`, it also returns the exit code of that program. When typing an entry by hand, end it with a single "`.`" or with Ctrl-D (Ctrl-Z on Windows).

4 Testing

`bash pd.sh test` compiles the four programs into a temporary directory and runs six groups of tests there, so the real diary is not changed. Each test compares the actual output with the expected output using `diff`, and the script stops at the first difference.

1. `pdadd`: input ended by "`.`" (the line after it is ignored) and input ended by EOF.
2. `pdlist`: date order, a range, and a range whose start and end are the same day.
3. Adding to the same date again replaces the old content.
4. Redirection: `pdshow > copy.txt`, then `pdadd < copy.txt` for another date.
5. `pdremove` succeeds once; removing the same date again fails with exit code 255.
6. Wrong input: an invalid date (`20260230`), a missing argument, and a start date after the end date.

The actual output is shown below. The messages under group 5 and group 6 are expected error messages from the failing commands.

```
$ bash pd.sh test
1. 添加：点号结束和文件末尾结束
2. 列表：日期排序和包含端点的范围
3. 同一天覆盖
4. 重定向协作：导出后添加到另一天
5. 删除成功，以及重复删除返回 255
No entry for this date
6. 错误输入（下面的报错是预期结果）
Invalid date
Please input like this : pdshow YYYYMMDD
Start date is after end date
All tests passed.
```

All tests passed, and the programs compiled without warnings under `-Wall -Wextra`.

5 Problems Encountered

Calling `parse` without an object. At first `parse` was a normal member function. When I called `Date::parse` in `Diary::load` to read a date line, the IDE reported "a nonstatic member reference must be relative to a specific object". The purpose of `parse` is to create a valid `Date`, so there is no `Date` object before it is called. I made it `static`; it returns whether the string is valid and writes the result into its second parameter.

Handling wrong data safely. Following suggestions from an AI assistant, I added more checks:

- `load` reads into a temporary map and only replaces the diary when the whole file is valid. A bad date line or an entry without the final "`.`" makes it fail, so a broken file is never half loaded.
- `pdadd` checks the state of `cin` after reading, and `save` checks the file stream after closing it.
- `pdadd` and `pdlist` open `diary.txt` in append mode once before loading. This creates the file on the first run without clearing an existing diary.

Testing the exit code of `pdremove`. `pdremove` returns -1, but the shell shows 255, because an exit code only keeps its lowest 8 bits. The test script therefore checks for 255. I also wrote the tests as `diff` comparisons in a temporary directory, so they can be repeated without checking the output by eye or touching the real diary.

Limitation. `pdshow` and `pdlist` also save the file at the end, as the assignment describes. When two programs are joined by a pipe, such as `pdshow A | pdadd B`, they run at the same time and may read and write `diary.txt` together, so this is not safe yet. The redirection test uses a temporary file instead. Saving only when the diary has changed would solve this.

6 Conclusion

The four programs share `Date`, `Entry` and `Diary`, keep all entries in one data file, and work together through redirection and `pd.sh`. Adding, listing, showing and removing entries all passed the tests.